

Shipay — Back-End Engineer Challenge

Questão 5 — API de criação de usuários

Objetivo	Implementar uma API REST em Python para criação de usuários, contemplando nome, e-mail, papel do usuário e senha opcional.
Stack	Python, FastAPI, Pydantic, SQLite, pytest e Ruff.
Validação final	9 testes automatizados passando; Ruff sem erros; arquivos formatados.

1. Solução proposta

A solução foi estruturada em camadas simples, evitando abstrações desnecessárias para o escopo da questão.

- FastAPI / Controller: recebe a requisição HTTP, utiliza o schema Pydantic, chama o Service e traduz erros de negócio para respostas HTTP.
- Pydantic Schema: valida os dados de entrada antes da execução das regras de negócio.
- UserService: concentra as regras de criação, incluindo validação do papel, geração de senha e proteção da senha.
- UserModel: encapsula as operações SQL para consultar o papel e persistir o usuário.
- SQLite: banco utilizado para uma implementação autocontida e facilmente executável no desafio.

Fluxo principal

POST /users → FastAPI → UserCreate → UserService → UserModel → SQLite → HTTP 201

Estrutura do projeto

```
question_5/
├── app/ (database.py, main.py, models.py, schemas.py, services.py)
├── data/ (schema.sql, shipay.db)
├── docs/
└── tests/
```

2. Regras de negócio

- Role existente: antes da criação, o Service verifica se o role_id informado existe em roles. Caso contrário, interrompe a operação com ValueError("Role not found").
- Senha fornecida: é utilizada para a criação, mas nunca é persistida em texto puro.
- Senha ausente: o Service gera automaticamente uma senha de 12 caracteres utilizando o módulo secrets.
- Proteção: a senha é transformada em hash com scrypt e salt aleatório de 16 bytes antes de chegar ao Model.
- Persistência: somente o hash da senha é enviado ao Model para gravação.

Fluxo da senha: senha recebida/gerada → salt aleatório → scrypt → hash → SQLite.

3. Modelo de dados e persistência

- users.role_id → roles.id por chave estrangeira.
- user_claims mantém a unicidade sobre (user_id, claim_id).
- A tabela claims mantém o campo active e a estrutura fornecida no desafio.
- O banco de desenvolvimento utilizado pela API é data/shipay.db.

O UserModel utiliza SQL parametrizado. Cada operação abre sua conexão, executa a operação necessária, realiza commit quando aplicável e garante o fechamento da conexão em finally.

4. API

POST /users — retorna 201 Created em caso de sucesso.

Campo	Tipo	Obrigatório	Regra
-------	------	-------------	-------

name	string	Sim	Nome do usuário.
email	EmailStr	Sim	Validação de formato pelo Pydantic.
role_id	int	Sim	Deve corresponder a um papel existente.
password	string None	Não	Se ausente, uma senha é gerada automaticamente.

Respostas de erro

- 422 Unprocessable Entity: falhas de validação do request, como ausência de campo obrigatório ou e-mail inválido, tratadas pelo Pydantic/FastAPI.
- 404 Not Found: utilizado quando o role_id informado não existe.

Exemplo de request

```
{ "name": "João Test", "email": "joao@test.com", "role_id": 2, "password": "MinhaSenha123" }
```

Injeção de dependência

O endpoint recebe o UserService por meio de Depends e Annotated. A função de dependência cria o Service com seu Model, mantendo o endpoint desacoplado dos detalhes de persistência.

5. Estratégia de testes

A suíte foi organizada em três níveis, permitindo validar separadamente persistência, regras de negócio e contrato HTTP.

Camada	Qtd.	Principais verificações
Model	2	Criação do usuário e existência de role.
Service	3	Senha fornecida, senha gerada e role inválido.
API	4	Criação, senha ausente, role inválido e e-mail ausente.
Total	9	Todos passando.

Isolamento dos testes

- Cada teste utiliza um banco SQLite temporário criado pelo tmp_path.
- O mesmo schema.sql da aplicação é reutilizado para criar a estrutura do banco de teste.
- Os testes da API usam dependency_overrides para injetar um UserService conectado ao banco temporário.
- O banco real de desenvolvimento não é utilizado pelos testes.

Execução: pytest -v — resultado final: 9 passed.

6. Qualidade e validação

- Ruff: ruff check app tests executado sem erros.
- Formatação: ruff format --check app tests sem arquivos pendentes.
- Testes: 9 testes executados com sucesso após as alterações finais.
- Segurança: senhas não são persistidas em plaintext; o banco recebe o resultado do scrypt com salt.
- SQL: comandos de inserção e consulta utilizam parâmetros.
- Escopo: a implementação evita abstrações adicionais que não são necessárias para o problema proposto.

Decisões de implementação

A solução deliberadamente mantém a arquitetura pequena: FastAPI → Service → Model → SQLite. O objetivo é demonstrar separação de responsabilidades, validação, regras de negócio, persistência e testes sem introduzir padrões ou componentes que não tragam benefício proporcional ao escopo da questão.

Observação sobre o ambiente de testes

Durante a execução da suíte, pode ser exibido um DeprecationWarning originado internamente pelo TestClient da versão utilizada de Starlette/AnyIO. O warning não é proveniente do código da aplicação e não interfere no resultado dos 9 testes.